

LAB – 8

AP23110011434

K.Tejasree

Section - N

Write a Prolog program to solve the “Water Jug Problem”.

```
move((X,Y),(4,Y)) :- X < 4.
```

```
move((X,Y),(X,3)) :- Y < 3.
```

```
move((X,Y),(0,Y)) :- X > 0.
```

```
move((X,Y),(X,0)) :- Y > 0.
```

```
move((X,Y),(X1,Y1)) :- X > 0, T is min(X,3-Y), X1 is X-T, Y1 is Y+T.
```

```
move((X,Y),(X1,Y1)) :- Y > 0, T is min(Y,4-X), Y1 is Y-T, X1 is X+T.
```

```
path(State,State,Visited,Visited).
```

```
path(State,Goal,Visited,Path) :-
```

```
    move(State,Next),
```

```
    \+ member(Next,Visited),
```

```
    path(Next,Goal,[Next|Visited],Path).
```

```
solve_waterjug(Path) :-
```

```
    path((0,0),(2,_),[(0,0)],Path).
```

 **solve_waterjug(Path).**

Path = [(2,0), (0,2), (4,2), (3,3), (3,0), (0,3), (4,3), (4,0), (0,0)]

Path = [(2,3), (2,0), (0,2), (4,2), (3,3), (3,0), (0,3), (4,3), (4,0), (0,0)]

Path = [(2,0), (0,2), (4,2), (3,3), (3,0), (0,3), (4,3), (1,3), (4,0), (0,0)]

Path = [(2,3), (2,0), (0,2), (4,2), (3,3), (3,0), (0,3), (4,3), (1,3), (4,0), (0,0)]

Write a program to implement a monkey banana problem.

```
action(state(middle,onbox,middle,hasnot), grasp, state(middle,onbox,middle,has)).
```

```
action(state(P,onfloor,P,H), climb, state(P,onbox,P,H)).
```

```
action(state(P1,onfloor,P1,H), push(P1,P2), state(P2,onfloor,P2,H)).
```

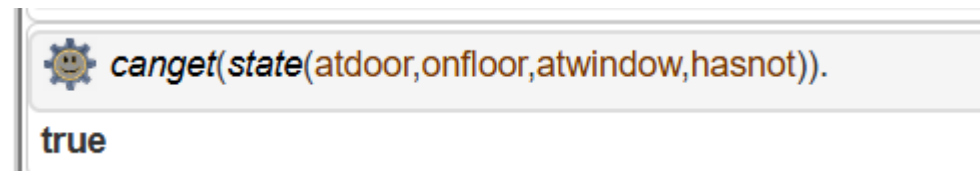
```
action(state(P,onfloor,B,H), walk(P,Q), state(Q,onfloor,B,H)).
```

```
canget(state(_,_,_,has)).
```

```
canget(State1) :-
```

```
    action(State1,_,State2),
```

```
    canget(State2).
```



Write a program to implement 8 Queens Problem.

```
queens(0, []).
```

```
queens(N, [Q|Qs]) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    queens(N1, Qs),
```

```
    member(Q, [1,2,3,4,5,6,7,8]),
```

```
    safe(Q, Qs, 1).
```

```
safe(_, [], _).
```

```
safe(Q, [Q1|Qs], D) :-
```

```
    Q \= Q1,
```

```
    abs(Q - Q1) \= D,
```

```
    D1 is D + 1,
```

```
    safe(Q, Qs, D1).
```



queens(8, Solution).

Solution = [4, 2, 7, 3, 6, 8, 5, 1]

Solution = [5, 2, 4, 7, 3, 8, 6, 1]

Write a program to solve travelling salesman problem.

edge(a,b,10). edge(a,c,15). edge(a,d,20).

edge(b,a,10). edge(b,c,35). edge(b,d,25).

edge(c,a,15). edge(c,b,35). edge(c,d,30).

edge(d,a,20). edge(d,b,25). edge(d,c,30).

path_tsp([],0).

path_tsp([X,Y|T],Cost) :-

edge(X,Y,C1),

path_tsp([Y|T],C2),

Cost is C1 + C2.

tsp(Path,Cost) :-

Path = [a|Rest],

permutation([b,c,d],Rest),

append(Path,[a],Cycle),

path_tsp(Cycle,Cost).



tsp(Path, Cost).

Cost = 95,

Path = [a, b, c, d]

Cost = 80,

Path = [a, b, d, c]